Category:

Reversing

Name:

Matryoshka is a Russian nested dolls.

Message:

You are given a zip file whose password is “CyberSeaGame”.

You will find an obfuscated huge PHP code inside it.

Decode it and you will get the flag.

Instructions:

You can decode it manually if you are very perseverant. But you aren’t if you have the “three virtues of Perl programmers” defined by Larry Wall. So you want to try decode in a programmatic way.

You may find some obfuscation methods used in this challenge.

- hex2bin
- base64_decode
- escape
- deflate
- gzcompress
- eval

There are open-sourced modules that allow us to enable hook PHP functions.

- evalhook (hosted in https://github.com/extremecoders-re/php-eval-hook)

- upoz (hosted in https://github.com/krakjoe/uopz)

But a naïve use of them only gives 2nd payload.

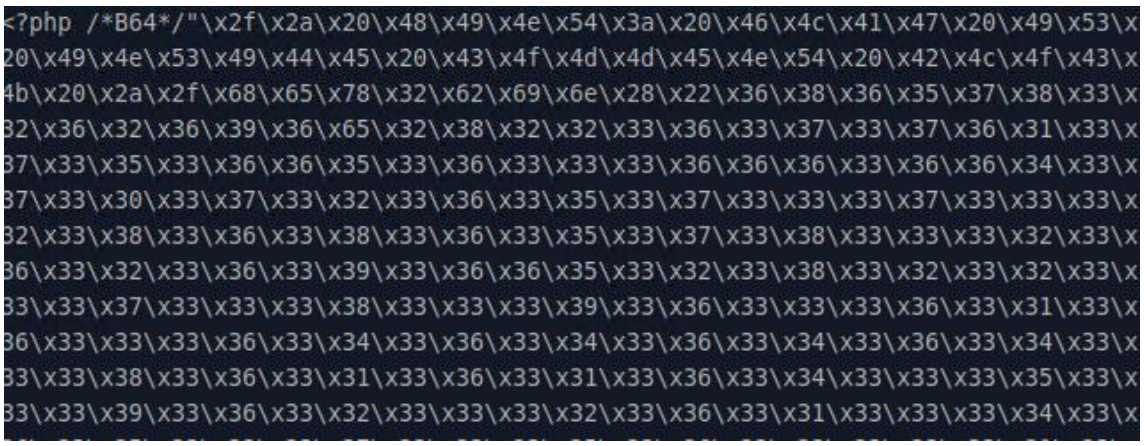
Example of evalhook output:

```
 eval() @ /home/ctrg/reverse-php-1/round_1.php(2) : eval()'d code:
 base64_decode("aGV4MmJpbigiNjc3YTY5NmU2NjZjNjE3NDY1Mjg2ODY1NzgzMjYyNjk2ZTI4MjIzMi
ijM0Mzk2MzM1NjYzOTM3NjIzMjMzMzg2NDM2NjMzNTYyNjY2MzYyMzc2MjMzMzUzNjYyNjUzNTY1MzIzOi
DMwNjY2MzM0MzkzNzMyMzUzOTM4NjEzMTM2MzI2MTM2NjEzNTM0MzUzNDYzNjEzOTYyMzczNzM4MzkzMi
TMyMzEzNTMxMzUzMTM1MzMzNTMwMzIzOTY2Mzc2NTMzMzY2MzY2NjE2MzM5NjU2NTM5NjU2NTYxMzczNi
DYyMzIzMTMyMzQ2NTM3NjU2MzY2NjQ2NDYyMzEzMDY1NjE2NDM4Mzk2MzMwNjIzNzY0NjQ2NDYyNjQ2Mi
jMzNjY2NjYxMzQ2MjYyMzUzNzY1MzY2MjYyNjY2NjM3NjY2MzM4NjYzNjY2MzczNDMzNjYzNjM5NjY2Ni
jY2MzY2MjM0MzY2MzY1NjIzNDMzNjMzMzYyNjQ2MjY2NjQ2MjM2NjQzMTY0MzUzNzMyNjEzMjMzNjEzOi
```

Then, we need to go to the next base64_decode, hex2bin, etc…
uopz_set_hook() gives us to intercept function calls.
Ex) uopz_set_hook('hex2bin', function($string){print $string;});
Implement hooks for all the methods….

Do not forget to add "<?php" and "?>" at the beginning of, and the end of the output to make them execute in PHP engine.

Decoding base64 and hex2bin will produce binary to pass *gzuncompress()* or *gzinflate().* Please take care of them.
In some cases, you need to unescaped strings, since no function is used.



You can add *print()* to this case.
Please take care to track the outermost function, for each obfuscation stage.

After you implement the decoder, you will get the flag.



CSG_FLAG{A_Matryoshka_Obfuscated_Script}

Spoiler) An example decoder from the challenge creator is as follows

harness.php (a PHP script to use uopz and evalhook plugin modules.)

```php
<?php
function is_bin($x){
    return preg_match('~[^\x20-\x7e\t\r\n]~', $x) > 0;
}
function __eval($code, $file) {
    print("<?php /*EVAL*/\n". $code . " ?>\n");
}

uopz_set_hook('base64_decode', function($string){
    $x = base64_decode($string);
    if(!is_bin($x)){
        if($x[0] == '"'){
            print "<?php /*B64*/ stripcslashes(". $x . "); ?>\n";
        }else{
            print "<?php /*B64*/" . $x . "; ?>\n";
        }
    }
});

uopz_set_hook('hex2bin', function($string){
    $x = hex2bin($string);
    if(!is_bin($x)){
        if($x[0] == '"'){
            print "<?php /*HEX*/ stripcslashes(". $x . "); ?>\n";
        }else{
            print "<?php /*HEX*/" . $x . "; ?>\n";
        }
    }
});

uopz_set_hook('json_decode', function($string){
    $x = json_decode($string);
});

uopz_set_hook('stripcslashes', function($string){
    $x = stripcslashes($string);
    if(!is_bin($x)){
        if(strchr($x, '\\') > 0){
            print "<?php /*SLASHES*/ print(\"" . $x . "\"; ?>\n";
        }else{
            print "<?php /*SLASHES*/ print(" . $x . "); ?>\n";
        }
    }
});

uopz_set_hook('gzinflate', function($string){
    $x = gzinflate($string);
    print "<?php /*INFLATE*/ " . $x . "; ?>\n";
});

uopz_set_hook('gzuncompress', function($string){
    $x = gzuncompress($string);
    print "<?php /*UNCOMPRESS*/ " . $x . "; ?>\n";
});

include($argv[1]);
?>
```

decode.sh (a runner script for harness.php)

```
#!/bin/bash -x

i=1
php harness.php round_1.php | grep -v 'You need to deobfuscate' | tail -n 1 > de
code_1.php

while true;
do
    n=`expr ${i} + 1`
    php harness.php decode_$i.php | grep "<\?php" | tail -n 1 > decode_$n.php
    if [ $? -ne 0 ]; then
        echo "Error in decode $i"
        exit 1
    fi
    y=`cat decode_$n.php | wc -l`
    if [ "$y" == "0" ]; then
        echo "Error in decode $i"
        exit 2
    fi
    x=`grep -i csg_flag decode_$n.php`
    if [ $? -eq 0 ]; then
        echo 'Found flag!'
        cat decode_$n.php
        exit 0
    fi

    i=${n}
done
```